

FFT Audio Spectrum Analyzer IP — Bản tiếng Việt

IP lõi phân tích phổ âm thanh dựa trên **FFT-256 R2SDF pipelined**, Q1.15, AXI4-Stream, mô phỏng. Lõi RTL đã kiểm chứng logic bằng mô hình Python cycle-accurate (bit-exact vs golden, SQNR ~60 dB, chống gap); **chưa biên dịch HDL trong môi trường tạo file** — hãy chạy testbench trên Vivado để xác nhận cú pháp/timing.

1. Tổng quan

- **Kiến trúc:** R2SDF DIF, N=256, fs=48 kHz, Q1.15 16-bit, throughput 1 mẫu/chu kỳ, output **bit-reversed** (được bin_ram_output sắp về tự nhiên, đơn biên N/2=128 bin).
- **Chuỗi:** audio (AXI-S) → input_buffer_fsm → window_unit (Hann) → fft_r2sdf_top → magnitude_unit (α -max- β -min) → bin_ram_output → peak_detector.
- **Bin** → **tần số:** $f = k \cdot fs / N$ (187.5 Hz/bin).

2. Trạng thái hoàn chỉnh (đối chiếu danh sách PDF §2)

Nhóm	Module	Trạng thái
Lõi FFT	butterfly_r2_stage, twiddle_rom, complex_multiplier, stage_with_twiddle, fft_r2sdf_top	✓ (đã sửa bug sel_o/cnt_o; Karatsuba 3-cyc; ROM đồng bộ; valid-gating)
Tiền/hậu xử lý	input_buffer_fsm, window_unit, window_rom, magnitude_unit, peak_detector	✓ (của nhóm)
Sắp xếp	bin_ram_output (magnitude), bit_reverse_reorder (phức, tùy chọn)	✓ (mới)
Giao diện IP	axi_stream_slave_if, axi_stream_master_if	✓ (mới, skid buffer)
Điều khiển	fft_control_fsm (status/enable)	✓ (mới)
Tích hợp	project_top, fft_analyzer_ip (IP top AXI-S 2 đầu)	✓
Python	golden_model_fft, gen_twiddle_rom, gen_window_coeff, gen_vectors, compare_sqnr	✓
Testbench	tb_fft_r2sdf (lõi), tb_top.sv (hệ thống)	✓
Đặc tả	interface_spec.md	✓

scaling_schedule_ctrl và convergent_rounding **tích hợp** (tham số SCALE_EN và hàm làm tròn trong từng module) — đúng chủ trương ghi trong PDF §2.

3. Điểm thiết kế then chốt

- **Delay-commutator butterfly** có delay line bên trong (DELAY_LEN=128...1); xuất `sel_o / cnt_o` (đã đăng ký để căn đúng với `out_re/out_im` = sửa bug W lệch pha).
- **Nhánh twiddle 4 chu kỳ** (ROM đồng bộ 1 + Karatsuba 3); **nhánh sum bypass khớp 4**; MUX theo `sel_o` trễ 4 → không lệch pha.
- **Scaling ÷2/tầng** (tổng 1/N) chống tràn Q1.15 → SQNR ~60 dB.
- **Valid-gating** (chân `en`) chịu bubble 1 chu kỳ giữa các frame của `input_buffer_fsm`.
- Độ trễ lỗi = **283 mẫu-valid** (đo bằng mô phỏng; testbench tự đo lại).

4. Build & mô phỏng trên Vivado

```
# sinh ROM/vector (nếu cần)
python3 python/gen_twiddle_rom.py --n 256 --out rom_out
python3 python/gen_window_coeff.py          # window_coeff.mem
python3 python/gen_vectors.py              # tv_*.mem
# kiểm lỗi FFT
iverilog -g2012 -o sim rtl/tb_fft_r22sdf.v rtl/fft_r22sdf_top.v rtl/stage_with_twiddle.v \
    rtl/butterfly_r2_stage.v rtl/complex_multiplier.v rtl/twiddle_rom.v && vvp sim
# kiểm hệ thống (Vivado xsim, SystemVerilog)
#   thêm tất cả rtl/*.v + tb_top.sv, đặt *.mem cùng thư mục sim, chạy tb_top
```

Kỳ vọng `tb_fft_r22sdf`: SQNR (RTL vs golden) rất cao, bin đỉnh=10, **PASS**, in latency thực (đặt `LATENCY / FFT_LATENCY` = con số này nếu khác 283). `tb_top`: peak_bin=10, **PASS**.

5. Ánh xạ kiểm chứng Verilog ↔ Python

```
complex_multiplier ↔ cmul_q15; twiddle_rom ↔ gen_twiddle_rom / twiddle_q15;
butterfly_r2_stage ↔ _Stage / round_sat_shift; fft_r22sdf_top ↔ fft_core_frame (SQNR);
magnitude_unit ↔ α-max-β-min; bin_ram_output ↔ bitrev; toàn chuỗi ↔ compare_sqnr.py.
```

6. Lưu ý

- Nếu bản `complex_multiplier` gốc không có chân `en`: (a) thêm `en`, hoặc (b) đảm bảo valid liên tục vào FFT (bỏ bubble). Nếu pipeline nhân ≠ 3 tầng → đổi số tầng bypass + LATENCY.
- Luồng phổ ra là self-timed; giữ `m_axis_tready` cao hoặc chèn FIFO trước `axi_stream_master_if`.

Xem `interface_spec.md` cho đặc tả cổng/định dạng đầy đủ.